will be matched with the corresponding method parameters. For example, you could use this approach for the following step:

```
Given I am a <status> Frequent Flyer member
```

Using this convention, the corresponding step definition could be written as follows:

```
[Given]
public void I_am_a_STATUS_Frequent_Flyer_member(string status)
{
    ...
}
```

6.7.5 Sharing data between steps

As you saw previously, it's often useful to be able to share data between steps during a scenario execution, even if the step definitions aren't in the same class. The SpecFlow scenario context provides a clean and convenient API that lets you do just that.

For example, you could store a Frequent Flyer member that you obtained from the database like this:

```
[Given(@"a frequent flyer member called {name}")]
public void GivenAFrequentFlyerMember(string name)
{
    var member = MemberHelper.FindByName(name);
    ScenarioContext.Current[name] = member;
}
```

Fetch member from database

Store member for later reference

This field is now available for future use in subsequent steps executed during this scenario. Here, for example, you fetch it in order to let the member make a flight booking:

```
[When(@"{name} books a flight")]
public void BooksAFlight(string name)
{
    var member ScenarioContext.Current[name];
    ...
}
```

Reuse the member you stored earlier.

6.7.6 Step definitions using example tables

If you need to pass a table of data to a step method, you can add a parameter of type `Table` to the step definition method. This class encapsulates the contents of the tabular parameter with a convenient API that lets you both extract data and compare tables.

Suppose you're writing a step definition for the following step:

```
Given the following accounts:
      | owner | points | statusPoints  |
      | Jill  | 100000 | 800           |
      | Joe   | 50000  | 50            |
```

You could do this in SpecFlow as follows:

```
[Given(@"the following accounts:")]
public void givenTheFollowingAccounts(Table accounts)
```